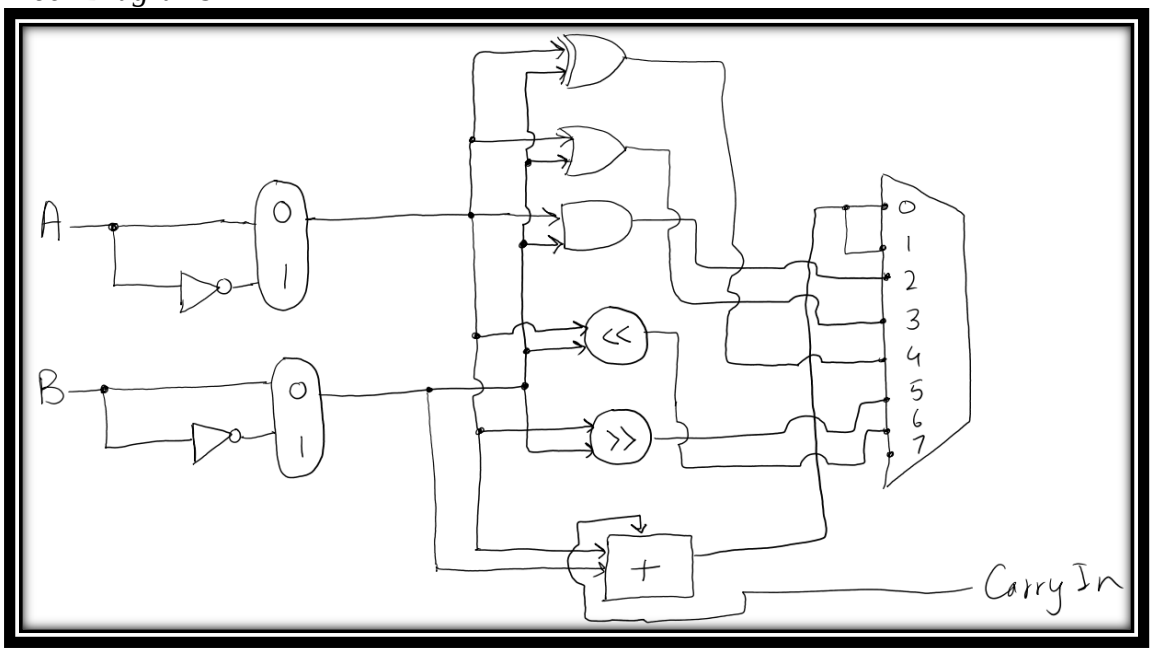
Lab 06 – Worksheet

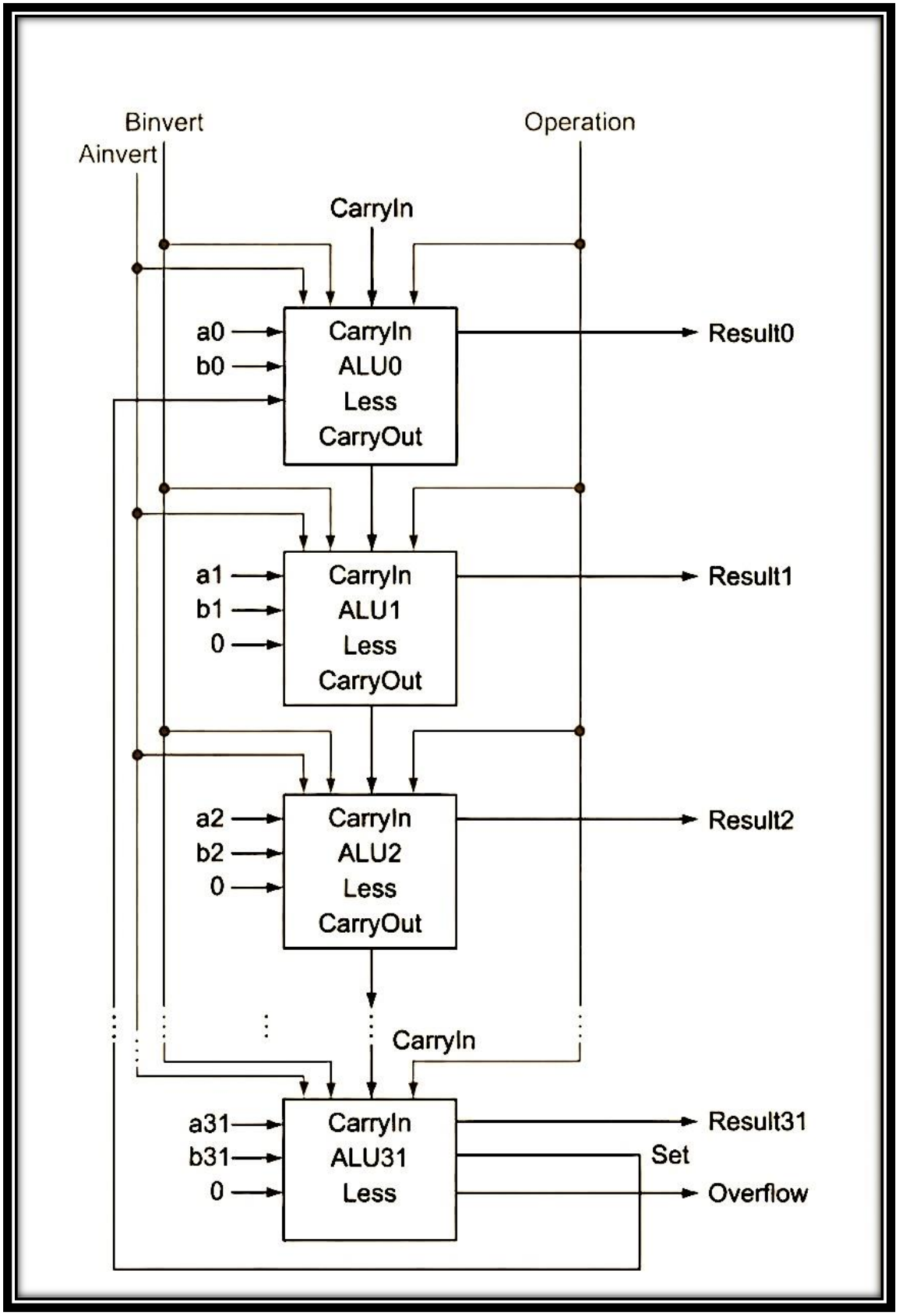
Name: Shaheer Qureshi, Areeba Izhar	ID: sq09647, ao09625	Section: T1

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Block Diagrams




Binvert
Ainvert
Operation
CarryIn
a0
b0
CarryIn
ALU0
Less
CarryOut
Result0
a1
b1
0
CarryIn
ALU1
Less
CarryOut
Result1
a2
b2
0
CarryIn
ALU2
Less
CarryOut
Result2
CarryIn
a31
b31
0
CarryIn
ALU31
Less
Result31
Set
Overflow

Comments:-

The first diagram is a 1 bit ALU with all different blocks of 7 operations that we are doing. In our ALU, the adder is doing the subtractor part as well and the 2-1 MUX on each decides which one to pass (0 for positive and 1 for negative) on the first two inputs of the final 8-1 MUX and the rest are mapped on to each value of the 8-1 final MUX for all the other standard gates; AND, OR, XOR, Logical Shift Right and Logical Shift Left.

2nd diagram is a 32 bit cascaded ALU made by cascading 32 1-bit ALUs in parallel by cascading, where carryout of one bit serves as the carryin for the next.

Task 2

Modules + Testbench:

Sources:-

1.

```
`timescale 1ns / 1ps
module alu_top (
    input wire clk,
    input wire pbin,                // Reset button
    input wire [15:0] physical_sw,    // FPGA Switches
    output wire [15:0] physical_leds  // FPGA LEDs
);
    wire rst_clean;
    debouncer rst_db (.clk(clk), .pbin(pbin), .pbout(rst_clean));
    wire [31:0] switch_data;
    leds switch_reader (
        .clk(clk), .rst(rst_clean), .btns(16'd0), .writeData(32'd0),
        .writeEnable(1'b0), .readEnable(1'b1), .memAddress(30'd0),
        .switches(physical_sw), .readData(switch_data)
    );
    reg [31:0] led_write_data = 32'd0;
    switches led_writer (
        .clk(clk), .rst(rst_clean), .writeData(led_write_data),
        .writeEnable(1'b1), .readEnable(1'b0), .memAddress(30'd0),
        .readData(), .leds(physical_leds)
    );
    wire [31:0] fixed_A = 32'h10101010;
    wire [31:0] fixed_B = 32'h01010101;
    reg  [3:0]  current_alu_ctrl;
    wire [31:0] alu_result;
    wire        alu_zero;
    alu_32bit ALU_Inst (
        .A(fixed_A),
```

```
    .B(fixed_B),
    .ALUControl(current_alu_ctrl),
    .ALUResult(alu_result),
    .Zero(alu_zero)
  );
  localparam READ_INPUTS = 1'b0;
  localparam UPDATE_LEDS = 1'b1;
  reg state = READ_INPUTS;
  always @(posedge clk) begin
    if (rst_clean) begin
      state <= READ_INPUTS;
      current_alu_ctrl <= 4'd0;
      led_write_data <= 32'd0;
    end else begin
      case (state)
        READ_INPUTS: begin
          // Read the bottom 4 switches to select the ALU operation
          current_alu_ctrl <= switch_data[3:0];
          state <= UPDATE_LEDS;
        end
        UPDATE_LEDS: begin
          // Format the LED output:
          // LED 15 = Zero Flag
          // LEDs 14:0 = Bottom 15 bits of the math result
          led_write_data <= {16'd0, alu_zero, alu_result[14:0]};
          state <= READ_INPUTS; // Loop back
        end
      endcase
    end
  end
endmodule
```

2,3,4.

debouncer.v, leds.v, switches.v used from Lab05.

5.

```
`timescale 1ns / 1ps
module alu_32bit(
    input  wire [31:0] A,
    input  wire [31:0] B,
    input  wire [3:0]  ALUControl,
    output reg  [31:0] ALUResult,
    output wire        Zero
);
    always @(*) begin
        case (ALUControl)
            4'b0000: ALUResult = A & B;          // AND
            4'b0001: ALUResult = A | B;          // OR
            4'b0010: ALUResult = A + B;          // ADD
            4'b0110: ALUResult = A - B;          // SUB
            4'b0100: ALUResult = A ^ B;          // XOR
            4'b1000: ALUResult = A << B[4:0];    // SLL (Shift Left Logical)
            4'b1001: ALUResult = A >> B[4:0];    // SRL (Shift Right Logical)
            default: ALUResult = 32'd0;          // Default to 0
        endcase
    end
    assign Zero = (ALUResult == 32'd0) ? 1'b1 : 1'b0;
endmodule
```

6.

```
`timescale 1ns / 1ps
module alu_1bit (
    input  wire      A,
    input  wire      B,
    input  wire [3:0] ALUControl,
    output reg       ALUResult,
    output wire      Zero
);
    always @(*) begin
        case (ALUControl)
            4'b0000: ALUResult = A & B;          // AND
            4'b0001: ALUResult = A | B;          // OR
            4'b0010: ALUResult = A ^ B;          // ADD
            4'b0110: ALUResult = A ^ B;          // SUB
            4'b0100: ALUResult = A ^ B;          // XOR
            4'b1000: ALUResult = 1'b0;           // SLL
            4'b1001: ALUResult = 1'b0;           // SRL
            default: ALUResult = 1'b0;
        endcase
    end
    assign Zero = ~ALUResult;
```

```
endmodule
```

Testbench files:-

1:-

```
`timescale 1ns / 1ps
module alu_1bit_tb;
   // Inputs
   reg A;
   reg B;
   reg [3:0] ALUControl;
   // Outputs
   wire ALUResult;
   wire Zero;
   // Instantiate the Unit Under Test (UUT)
   alu_1bit uut (
      .A(A),
      .B(B),
      .ALUControl(ALUControl),
      .ALUResult(ALUResult),
      .Zero(Zero)
   );
   initial begin
      // Initialize Inputs
      A = 0;
      B = 0;
      ALUControl = 0;
      // Wait 10 ns for global reset to finish
      #10;
      // Print header for the console output
      // Test 1: AND
      ALUControl = 4'b0000; A = 1'b1; B = 1'b1; #10;
      ALUControl = 4'b0000; A = 1'b1; B = 1'b0; #10;
      // Test 2: OR
      ALUControl = 4'b0001; A = 1'b0; B = 1'b1; #10;
      ALUControl = 4'b0001; A = 1'b0; B = 1'b0; #10;
      // Test 3: ADD (Acts as XOR in this dataless 1-bit version)
      ALUControl = 4'b0010; A = 1'b1; B = 1'b0; #10;
      ALUControl = 4'b0010; A = 1'b1; B = 1'b1; #10; // Result should be 0, Zero flag
1
      // Test 4: SUB (Acts as XOR)
      ALUControl = 4'b0110; A = 1'b1; B = 1'b1; #10; // Result should be 0, Zero flag
1
      ALUControl = 4'b0110; A = 1'b0; B = 1'b1; #10;
      // Test 5: XOR
      ALUControl = 4'b0100; A = 1'b1; B = 1'b0; #10;
      // Test 6: SLL (Expected to output 0 as per our 1-bit module)
```

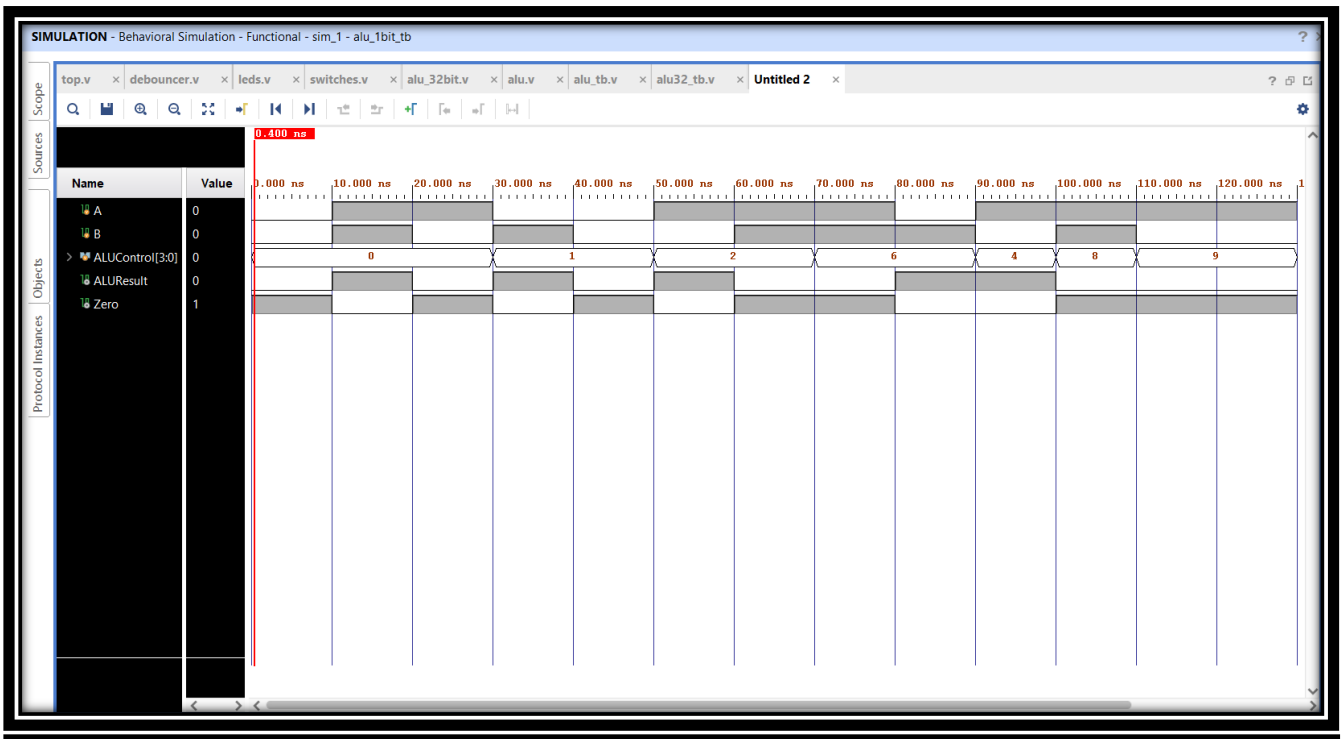
```
        ALUControl = 4'b1000; A = 1'b1; B = 1'b1; #10;
        // Test 7: SRL (Expected to output 0)
        ALUControl = 4'b1001; A = 1'b1; B = 1'b0; #10;
        // End simulation
        #10;
        $finish;
    end
endmodule
```

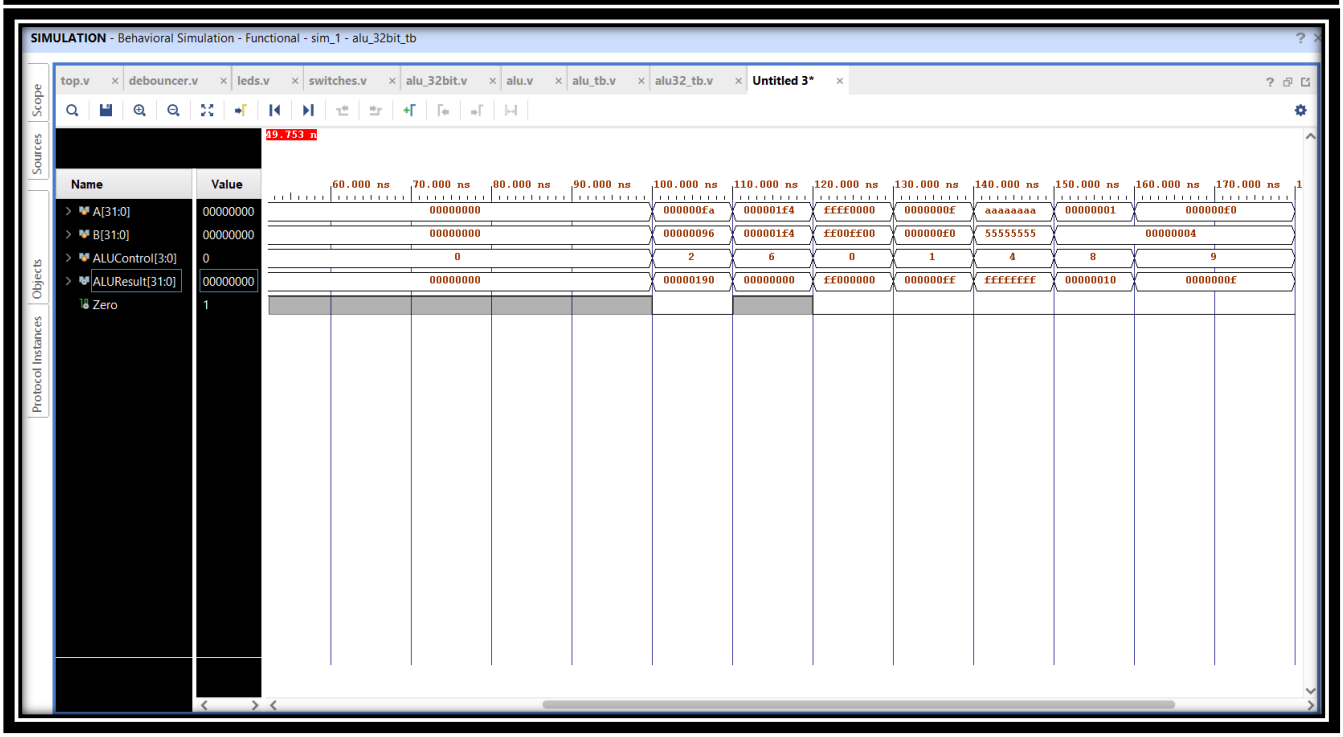
2:-

```
`timescale 1ns / 1ps
module alu_32bit_tb;
    // Inputs
    reg [31:0] A;
    reg [31:0] B;
    reg [3:0]  ALUControl;
    // Outputs
    wire [31:0] ALUResult;
    wire        Zero;
    // Instantiate the Unit Under Test (UUT)
    alu_32bit uut (
        .A(A),
        .B(B),
        .ALUControl(ALUControl),
        .ALUResult(ALUResult),
        .Zero(Zero)
    );
    initial begin
        // Initialize Inputs
        A = 0;
        B = 0;
        ALUControl = 0;
        // Wait 100 ns for global reset to finish
        #100;
        // Test 1: ADD
        A = 32'd250;
        B = 32'd150;
        ALUControl = 4'b0010; // ADD
        #10;
        // Test 2: SUB
        A = 32'd500;
        B = 32'd500;
        ALUControl = 4'b0110; // SUB
        #10;
        // Test 3: AND
        A = 32'hFFFF_0000;
        B = 32'hFF00_FF00;
```

```
        ALUControl = 4'b0000; // AND
        #10;
        // Test 4: OR
        A = 32'h0000_000F;
        B = 32'h0000_00F0;
        ALUControl = 4'b0001; // OR
        #10;
        // Test 5: XOR
        A = 32'hAAAA_AAAA;
        B = 32'h5555_5555;
        ALUControl = 4'b0100; // XOR
        #10;
        // Test 6: SLL (Shift Left Logical)
        A = 32'h0000_0001;
        B = 32'd4; // Shift left by 4
        ALUControl = 4'b1000; // SLL
        #10;
        // Test 7: SRL (Shift Right Logical)
        A = 32'h0000_00F0;
        B = 32'd4; // Shift right by 4
        ALUControl = 4'b1001; // SRL
        #10;
        // End simulation
        #10;
        $finish;
    end
endmodule
```

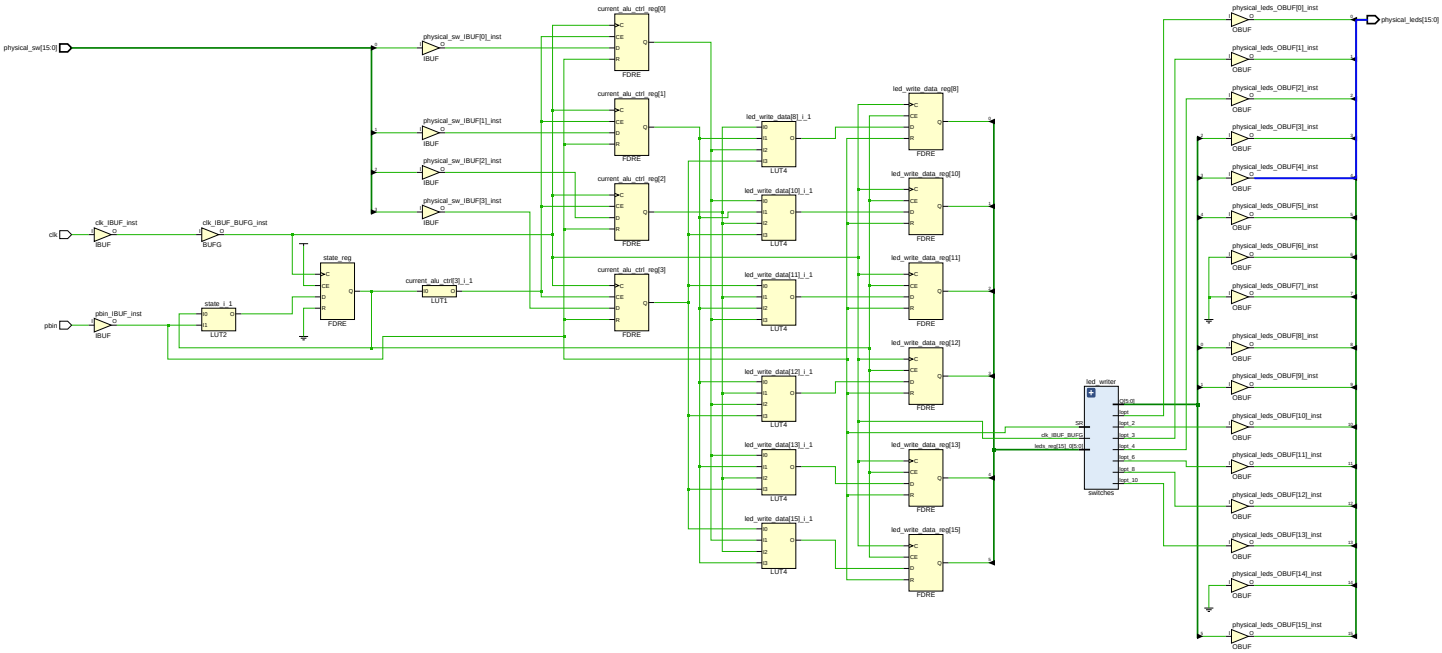
Results:





Task 3

RTL Schematics


physical_sw[15:0]
clk
pbin
clk_IBUF_inst
IBUF
clk_IBUF_BUFG_inst
BUFG
pbin_IBUF_inst
IBUF
state_i_1
LUT2
state_reg
FDRE
current_alu_ctrl[3]_i_1
LUT1
physical_sw_IBUF[0]_inst
physical_sw_IBUF[1]_inst
physical_sw_IBUF[2]_inst
physical_sw_IBUF[3]_inst
current_alu_ctrl_reg[0]
current_alu_ctrl_reg[1]
current_alu_ctrl_reg[2]
current_alu_ctrl_reg[3]
led_write_data[8]_i_1
led_write_data[10]_i_1
led_write_data[11]_i_1
led_write_data[12]_i_1
led_write_data[13]_i_1
led_write_data[15]_i_1
LUT4
led_write_data_reg[8]
led_write_data_reg[10]
led_write_data_reg[11]
led_write_data_reg[12]
led_write_data_reg[13]
led_write_data_reg[15]
led_writter
switches
clk_IBUF_BUFG
leds_reg[15]_0[0:3]
physical_leds_OBUF[0]_inst
physical_leds_OBUF[1]_inst
physical_leds_OBUF[2]_inst
physical_leds_OBUF[3]_inst
physical_leds_OBUF[4]_inst
physical_leds_OBUF[5]_inst
physical_leds_OBUF[6]_inst
physical_leds_OBUF[7]_inst
physical_leds_OBUF[8]_inst
physical_leds_OBUF[9]_inst
physical_leds_OBUF[10]_inst
physical_leds_OBUF[11]_inst
physical_leds_OBUF[12]_inst
physical_leds_OBUF[13]_inst
physical_leds_OBUF[14]_inst
physical_leds_OBUF[15]_inst
OBUF
physical_leds[15:0]

Lab 05 – Designing FSM Using FPGA Switches and LEDs
Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	**Program/Code/ Simulation Model/ Network Model**	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR 5	**Results & Plots**	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	**Report**	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.